Rapport på løsning Oblig 3 – IN2031

Implementasjon av Events

Matias Nupen matiasnu@ifi.uio.no

2. november 2025

1. Introduksjon

Denne rapporten dokumenterer løsningen på den tredje obligatoriske oppgaven i IN2031. Oppgaven er tredje del av fire, som samlet skal resultere i et domene-spesifikt-språk (DSL). Jeg bruker hver oppgave som et delmål i prosjektet, med GitHub som versjonskontroll og utviklingsbrancher for å strukturere arbeidet. Koden i oblig 3 er basert på min tidligere løsning fra oblig 2, men noen klasser som `REPL.java` er hentet og endret fra prekoden som ble gitt ut.

Målet med denne tredje delen er å utvide språket med muligheten til å håndtere Events, og implementere en løsning som lar brukeren kunne interagere med programmet. Tidligere leste programmet inn en `.aero` fil som ble kjørt og logget. Denne implementasjonen utvider prosjektet og deler inn en `Statement` inn i `Reaction` og `Action`, hvor `Action` ble utviklet i forrige oppgave, implementeres `Reaction` i denne løsningen. En `Reaction` vil alltid være et `Event`, og det er blitt implementert tre typer `Event`: `Obstacle`, `Low_battery` og `Message [id]`. Rapporten redegjør for implementasjonen, hvordan jeg testet løsningen, og hvilke utfordringer og mulige forbedringer som ble identifisert.

2. Implementasjon

Som i oblig 1 og 2, er programmet strukturert rundt en ANTLR-bygd parser og lexer, en `AST`, og en `Interpreter` som traverserer treet.

3. Grammatikk og språkutvidelse

Grammatikken som skulle utvide programmet ble gitt i oppgaven, og er definert i `AeroScript.g4`. Det ble implementert ny syntaks for `Reaction` og `Event`, og definisjoner av tre typer `Event`: `Obstacle`, `Low_battery` og `Message [id]`. `Statement` ble også utvidet til å inkludere både `Reaction` og `Action`.

3.1. AST-utvidelse

Det ble implementert klasser basert på samme arveprinsipp som `Action`-klassene fra oblig 2, hvor hver type `Event` arver fra den abstrakte `Event`-klassen. Disse klassene har enkle metoder som

forteller noe om de er aktive og diverse hentemetoder for aktuelle verdier de beholder. `MessageEvent` beholder en `messageId`. I tillegg ble det implementert en klasse `Reaction` som beholder `targetModeName` og arver fra `Statement`-klassen.

3.2. Mode-håndtering

For å større den nye event-drevne arkitekturen ble klassen `Mode` utvidet for å representere de ulike tilstandene programmet kan være i. Som tidligere fungerer en mode som en beholder for statements og definerer programmets oppførsel i en gitt kontekst. Klassen ble utvidet med metoden `checkReactions(DroneState)`, som itererer gjennom alle statements og sjekker om noen reactions er triggered basert på nåværende `DroneState`. Hvis er reaction trigges, returnerer metoden target mode-navnet slik at `Interpreter` kan gjennomføre overgangen.

For å støtte de nye konstruksjonene måtte klassehierarkiet for statements utvides. `Statement` forble som abstrakt superklasse med `execute()` -og `toString()` metoder, mens Action fortsatte å representere utførbare handlinger som utviklet i oblig 2. Det nye elementet var Reaction, som representerer event-baserte overganger mellom modes. En `Reaction` består av et `Event`-objekt som definerer hva som trigger reaksjonen, og en `targetModeName` som angir hvilken mode som skal aktiveres ved triggering. Metoden `isTriggered(DroneState)` delegerer Event-objektets tilsvarende metode for å sjekke om event-betingelsen er oppfylt. `execute()`-metoden er tom, da reaksjoner ikke utføres direkte, men trigger mode-transitions som håndteres av `Interpreter`.

3.3. Typeklassen Obstacle

For å håndtere obstacle-events i AeroScript, ble det implementert en ny klasse i type-mappen, `Obstacle.java` som representerer hindringer i dronens navigasjonsområde. En Obstacle defineres av tre attributter: `obstacleName` som identifikator, `location` som et Point-objekt, og `isDiscovered` som boolean-flagg for å markere om hindringen er oppdaget. I implementasjonen antas det at en hindring kun tar opp ett nytt punkt i rommet uten dimensjoner. `DroneState` holder en liste over detekterte hindringer og tilbyr metoder som `discoverObstacle(Obstacle)` for å legge til hindringer, `hasObstacleOnPoint(Point)` for å sjekke om et punkt er blokkert, og `hasObstacleOnMoveByDistance(Point, Point)` som sjekker om det ligger hindringer langs en bevegelsessti ved å hente alle punkter med `0.1f` enhets intervall. `obstacleDetected()` returnerer true hvis det finnes oppdagede hindringer, og brukes av `ObstacleEvent` til å trigge tilhørende reaction som fører til mode-transitions for å håndtere situasjonen.

3.4. REPL-implementasjon

REPL-funksjonaliteten ble gitt ut som prekode med en grunnleggende struktur, men måtte utvides for å støtte implementasjonen. Prekoden innheoldt en Command-klasse for kommandohåndtering og fire

basiskommandoer: help, message, info og exit. Den opprinnelige message-kommandoen hadde kun en enkel implementasjon som sjekket om en listener eksisterte og kalte `receiveMessage()` på `Mode`-objektet.

Message-kommandoen ble utvidet til å ikke bare trigge listeners, men også legge til meldingen i `DroneState` ved hjelp av `DroneState.getInstance().addReceivedMessage(param)`. Dette sikrer at `MessageEvent.isTriggered()` kan sjekke om en melding er mottatt og trigge tilhørende reactions.

For å gi brukeren bedre innsikt og kontroll over systemet, ble det implementert flere nye kommandoer. `status`-kommandoen viser en detaljert oversikt over dronens nåværende tilstand, inkludert posisjon, høyde, batterinivå, distanse og aktiv mode, presentert i et oversiktelig tabell `messages`-kommandoen lister alle tilgjengelige message listeners som er registrert i programmet, noe som gjør det enklere for brukeren å vite hvilke meldinger som kan sendes. `obstacles`-kommandoen viser alle oppdagede hindringer i systemet, mens `obstacle`-kommandoen lar brukeren lage en hindring ved å oppgi koordinater. Denne kommandoen oppretter et `Obstacle`-objekt og legger det til i DroneState, noe som også vil trigge `ObstacleEvent`.

REPL-implementasjonen integreres i programflyten etter at `initializeProgram()` har kjørt start-mode. Hvis start-mode kun inneholder reactions uten actions, eller når alle actions i start-mode er fullført, går programmet over i REPL-loop hvor det venter på brukerkommandoer. Dette muliggjør interaktiv kontroll av dronen gjennom hele kjøringen, hvor brukeren kan sende meldinger, simulere hendelser og overvåke systemtilstand i sanntid.

3.5. DroneState-utvidelser

DroneState-klassen ble utvidet med to nye lister for event-håndtering: `detectedObstacles` for hindringer og `messageIds` for mottatte meldinger. For hindringshåndtering ble det implementert metoder som `discoverObstacle()` for å legge til hindringer, `hasObstacleOnPoint()` og `hasObstacleOnMoveByDistance()` for kolligsjondeteksjon, og `obstacleDetected()` som brukes av `ObstacleEvent`. For meldinger ble `addReceivedMessage()`, `removeMessageId()` og `messageReceived()` implementert, hvor sistnevnte kalles av `MessageEvent.isTriggered()`. I tillegg ble `batteryIsLow()` implementert for å støtte `LowBatteryEvent` ved å returnere true når batterinivået er 20% eller lavere. For testing ble `resetForTesting()` lagt til for å nullstille tilstand mellom testesekvenser.

3.6. Interpreter-utvidelser og programflyt

Interpreter ble betydelig utvidet i `executeMode()`-metoden som nå også håndterer event-drevet mode-kjøring. Metoden skiller mellom modes med kun reactions, som venter passivt på events, og modes med actions som utføres sekvensielt mens `checkReactions(DroneState)` sjekker

kontinuerlig for triggered reactions. Obstacle events sjekkes før hver action for umiddelbar respons på om dronen treffer en obstacle som er detected, mens battery- og message events sjekkes etterpå. Når en reaction trigges, avbrytes gjeldende mode umiddelbart og gjenværende actions forkastes før programmet hopper til target mode. `initializeProgram()` erstatter direkte kjøring og setter opp programmet for REPL-interaksjon ved å identifisere start-mode basert på `isInitial()`-flagget, initialisere `DroeState`, og kalle `buildListeners()` som registrerer alle message events. Programflyten starter med parsing av .aero-filen, etterfulgt av initialisering og eventuell kjøring av start-mode, før programmet går over i REPL-loop hvor det venter på brukerkommandoer til `exit` skrives.

4. Testing

Testsuiten fra oblig 2 ble beholdt for å verifisere at eksisterende funksjonalitet fortsatt fungerer korrekt. For den nye event-funksjonaliteten i oblig 3 ble det implementert tester for event-håndtering og mode-transitions. To av testene verifiserer at reactions trigges korrekt når tilhørende event oppstår, og at mode-transitions håndteres som forventet. `DroneState.resetForTesting()`-metoden ble implementert spesifikt for å sikre en ren tilstand mellom testsekvenser. REPL-funksjonaliteten er primært manuelt testet gjennom kjøring av `.aero`-programmer, da brukerinteraksjon er utfordrende å automatisere med `JUnit`.

5. Utfordringer og eventuelle problemer

Den største utfordringer var å håndtere timing av event-sjekkingg, spesielt at obstacle events måtte sjekkes før action-utførelse mens andre events sjekkes etterpå. REPL-integrasjonen krevde håndtering av asynkrone message events samtidig som Interpreter kjører modes. Potensielle forbedringer inkluderer mer robust feilhåndtering for manglende target modes.

6. Konklusjon

Denne oppgaven har utviklet AeroScript fra et sekvensielt skriptspråk til et event-drevet DSL med interaktiv brukerkontroll gjennom implementering av Reaction-systemet, tre event-typer og REPL-funksjonalitet. Løsningen bygger naturlig videre på funksjonaliteten fra oblig 2 og muliggjør dynamisk respons på hendelsene: obstacle, low battery og message under kjøring.